FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Crisostomo, Jerome M.
Section:	AN42
Professor:	Ma'am. Crisola Tan
Date:	10/02/2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.
Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking
✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ "New case saved" or similar confirmation
✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

```
import json

# ============================================================
# 1. Define the Case Base (Past Student Interventions)
# ============================================================
case_base = [
    {
        "id": "CASE_101",
        "problem": {
            "topic": "Algebra",
            "error_type": "Formula Misuse",
            "error_frequency": 3,
            "skill_level": "Beginner"
        },
        "solution": {
                    "feedback": "Review the quadratic formula components
step-by-step.",
            "recommended_lesson": "Algebra Fundamentals: Video Walkthrough
2.1",
            "practice_difficulty": "Easy",
            "recommended_practice_problems": 3
        }
    },
```

```
    {
        "id": "CASE_102",
        "problem": {
            "topic": "Calculus",
            "error_type": "Sign Error",
            "error_frequency": 2,
            "skill_level": "Intermediate"
        },
        "solution": {
             "feedback": "Pay close attention when applying the chain rule
to negative exponents.",
             "recommended_lesson": "Derivatives: Signs & Negative Exponent
Drill",
            "practice_difficulty": "Medium",
            "recommended_practice_problems": 4
        }
    },
    {
        "id": "CASE_103",
        "problem": {
            "topic": "Algebra",
            "error_type": "Formula Misuse",
            "error_frequency": 5,
            "skill_level": "Intermediate"
        },
        "solution": {
              "feedback": "Double check parenthetical distribution before
substituting values into the formula.",
               "recommended_lesson": "Algebra: Common Formula Pitfalls &
Practice Modules",
            "practice_difficulty": "Medium",
            "recommended_practice_problems": 5
        }
    },
    {
        "id": "CASE_104",
        "problem": {
```

```
            "topic": "Geometry",
            "error_type": "Conceptual Misunderstanding",
            "error_frequency": 4,
            "skill_level": "Beginner"
        },
        "solution": {
                "feedback": "Focus on the geometric definitions of angles
before applying the theorems.",
            "recommended_lesson": "Geometry Foundations: Interactive Angle
Visualizer",
            "practice_difficulty": "Easy",
            "recommended_practice_problems": 3
        }
    }
]


# ================================================================
# 2. Similarity Assessment (Weighted Scoring)
# ================================================================
def calculate_similarity(query_problem, case_problem):
    """
    Computes a weighted similarity score (0.0 to 1.0):
    - Topic match: 35% weight
    - Error Type match: 35% weight
    - Skill Level match: 15% weight
    - Error Frequency proximity: 15% weight (max difference span: 5)
    """
        topic_score = 1.0 if query_problem["topic"].strip().lower() ==
case_problem["topic"].lower() else 0.0
     error_score = 1.0 if query_problem["error_type"].strip().lower() ==
case_problem["error_type"].lower() else 0.0
     skill_score = 1.0 if query_problem["skill_level"].strip().lower() ==
case_problem["skill_level"].lower() else 0.0

                freq_diff = abs(query_problem["error_frequency"] -
case_problem["error_frequency"])
```

```
    freq_score = max(0.0, 1.0 - (freq_diff / 5.0))

    total_similarity = (
        (topic_score * 0.35) +
        (error_score * 0.35) +
        (skill_score * 0.15) +
        (freq_score * 0.15)
    )
    return round(total_similarity, 3)

def retrieve_best_case(new_problem, cases):
    ranked_cases = []
    for case in cases:
        score = calculate_similarity(new_problem, case["problem"])
        ranked_cases.append((case, score))

    # Sort descending by similarity score
    ranked_cases.sort(key=lambda x: x[1], reverse=True)
    return ranked_cases[0], ranked_cases


# ================================================================
# 3. Adapt the Solution
# ================================================================
def adapt_solution(retrieved_case, new_problem, student_name="Student"):
    base_solution = retrieved_case["solution"]
    adapted = dict(base_solution)

    # Personalize feedback message
            adapted["feedback"]   =   f"Hi   {student_name},   we   noticed
{new_problem['error_frequency']}                instances                of
'{new_problem['error_type']}'           in           {new_problem['topic']}.
{base_solution['feedback']}"

    # Scale problem count and adapt difficulty
    if new_problem["error_frequency"] >= 4:
```

```
                                adapted["recommended_practice_problems"]    =
max(base_solution["recommended_practice_problems"], 6)
        adapted["practice_difficulty"] = "Guided Step-by-Step"
    else:
                        adapted["recommended_practice_problems"]   =   max(3,
new_problem["error_frequency"] + 1)
                                            adapted["practice_difficulty"]     =
new_problem["skill_level"].capitalize()

                    adapted["recommended_lesson"]     =     f"[Adapted     for
{new_problem['skill_level'].capitalize()}]
{base_solution['recommended_lesson']}"
    return adapted


# ============================================================
# 4. Retain Stage (Learning)
# ============================================================
def retain_case(cases, new_problem, final_solution):
    new_id = f"CASE_{101 + len(cases)}"
    new_case_entry = {
        "id": new_id,
        "problem": new_problem,
        "solution": final_solution
    }
    cases.append(new_case_entry)
    return new_case_entry


# ============================================================
# 5. Interactive Execution
# ============================================================
if __name__ == "__main__":
    print("=" * 65)
    print("      INTELLIGENT TUTORING SYSTEM (ITS) — STUDENT INPUT")
    print("=" * 65)
```

```
    # Interactive User Inputs
    student_name = input("Enter student name: ").strip() or "Student"

    print("\nAvailable Topics: Algebra, Calculus, Geometry")
    topic = input("Enter topic: ").strip().capitalize()

       print("\nCommon  Errors:  Formula  Misuse,  Sign  Error,  Conceptual
Misunderstanding")
    error_type = input("Enter error type: ").strip().title()

    while True:
        try:
             error_frequency = int(input("Enter error frequency (number of
occurrences): ").strip())
            if error_frequency > 0:
                break
            print("Please enter a positive number.")
        except ValueError:
            print("Invalid input. Please enter an integer.")

    print("\nSkill Levels: Beginner, Intermediate, Advanced")
    skill_level = input("Enter skill level: ").strip().capitalize()

    new_student_case = {
        "topic": topic,
        "error_type": error_type,
        "error_frequency": error_frequency,
        "skill_level": skill_level
    }

    # Step A: Problem Description Display
    print("\n" + "=" * 65)
    print("[1] NEW PROBLEM SUBMISSION:")
    print(f"Student Name    : {student_name}")
    print(f"Topic           : {new_student_case['topic']}")
    print(f"Detected Error  : {new_student_case['error_type']}")
```

```
        print(f"Error  Frequency  :  {new_student_case['error_frequency']}
times")
    print(f"Skill Level     : {new_student_case['skill_level']}")

    # Step B: Retrieval & Similarity Assessment
                        (best_case,      top_score),      all_ranked      =
retrieve_best_case(new_student_case, case_base)

    print("\n[2] SIMILARITY RANKING OF PAST CASES:")
    for rank, (case, score) in enumerate(all_ranked, 1):
                         print(f"      {rank}.   {case['id']}   |   Topic:
{case['problem']['topic']:<8} | Error: {case['problem']['error_type']:<26}
| Similarity: {score * 100:.1f}%")

      print(f"\n✔ Most Relevant Past Case Retrieved: {best_case['id']}
(Match: {top_score * 100:.1f}%)")
    print("   Problem Profile:", json.dumps(best_case["problem"]))
    print("   Original Solution:", json.dumps(best_case["solution"]))

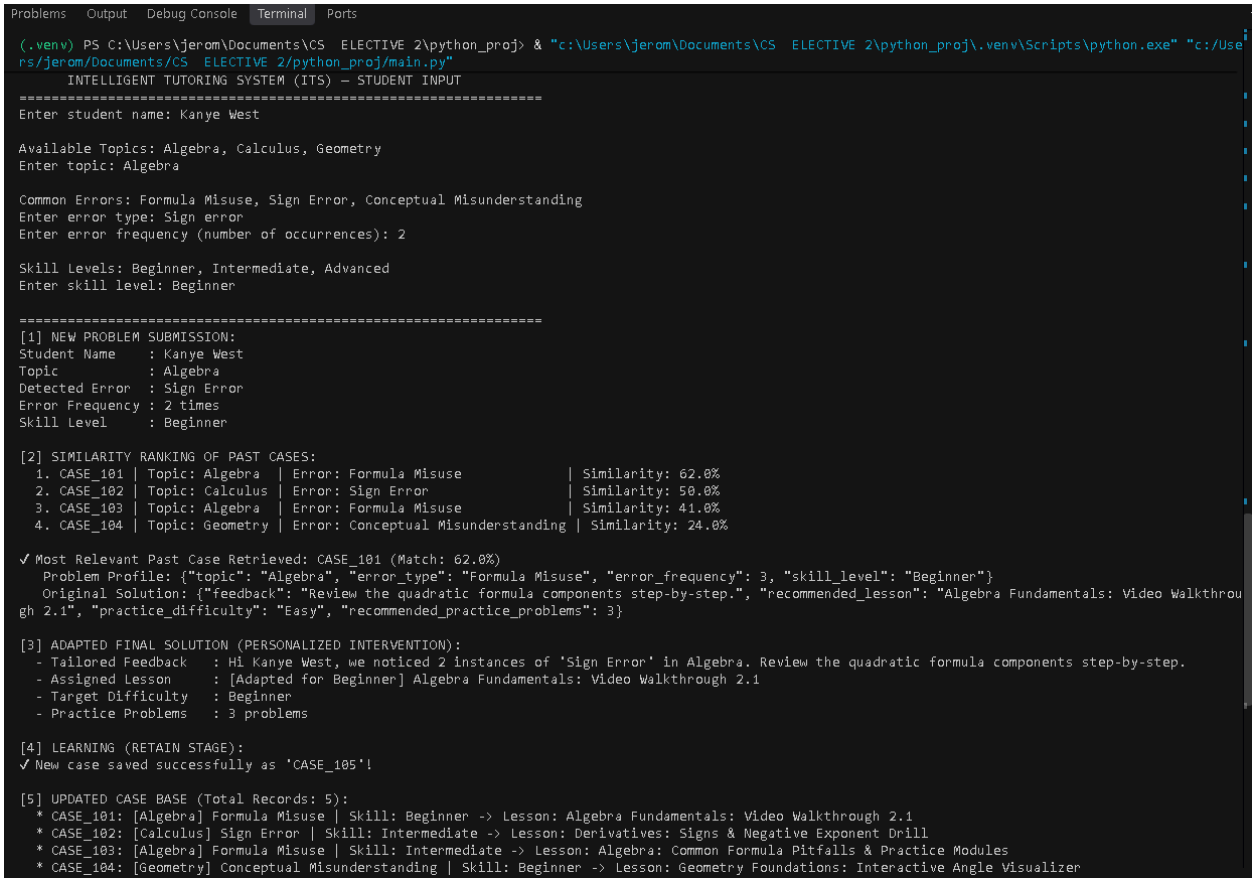
    # Step C: Adaptation
        adapted_solution  =  adapt_solution(best_case,  new_student_case,
student_name=student_name)

    print("\n[3] ADAPTED FINAL SOLUTION (PERSONALIZED INTERVENTION):")
    print(f"  - Tailored Feedback   : {adapted_solution['feedback']}")
              print(f"       -    Assigned    Lesson                  :
{adapted_solution['recommended_lesson']}")
                print(f"       -    Target    Difficulty              :
{adapted_solution['practice_difficulty']}")
                print(f"       -    Practice    Problems              :
{adapted_solution['recommended_practice_problems']} problems")

    # Step D: Retain (Learning)
            saved_case    =    retain_case(case_base,    new_student_case,
adapted_solution)
    print(f"\n[4] LEARNING (RETAIN STAGE):")
    print(f"✔ New case saved successfully as '{saved_case['id']}'!")
```

```
    # Step E: Updated Case Base Overview
    print(f"\n[5] UPDATED CASE BASE (Total Records: {len(case_base)}):")
    for case in case_base:
                print(f"    *  {case['id']}:  [{case['problem']['topic']}]
{case['problem']['error_type']}  |  Skill:  {case['problem']['skill_level']}
-> Lesson: {case['solution']['recommended_lesson']}")
    print("=" * 65)
```

Output:



B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study: Case Study 3: Intelligent Tutoring System
- Description of your case base: In this Intelligent Tutoring System, the base case acts as the system's foundational memory, storing historical records of prior student struggles paired with the successful educational solutions that resolved them.

- Explanation of similarity method: The similarity assessment relies on a multi-attribute weighted scoring model that quantifies how closely an incoming student's diagnostic profile aligns with prior records across categorical and numerical dimensions. Categorical attributes—specifically the mathematical topic 35% and error classification 35% are prioritized as primary domain indicators using binary exact-matching, while learner competency 15\% accounts for baseline proficiency. To evaluate error recurrence, numerical error frequency 15% is measured through normalized linear distance, which gracefully penalizes deviations between the two cases rather than enforcing a rigid binary cutoff. By taking the weighted sum of these sub-scores, the algorithm assigns each candidate case an aggregate similarity rating between 0.0 and 0.1,, sorting the historical repository in descending order to reliably surface the single most pedagogically relevant case for solution adaptation.
- Adaptation logic: The adaptation logic bridges the gap between historical precedent and the individual student's immediate needs by dynamically recalibrating the retrieved solution across four key dimensions: messaging, difficulty, workload, and curricular targeting

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed:
- What challenges you encountered
- What you learned about CBR

I didn't have a pair for this technical assessment but I've got to admit that I finished the activity with the help of Gemini. They helped me create the base case of the Intelligent Tutoring System and similarity system; I then proceeded to complete the project with the implementation of the solution system and the retain stage.

The challenges I encountered for this project is making sure that the tutoring system can accurately compare the new case with the base cases. Another is designing a meaningful similarity metric eg. deciding how much weight to give categorical attributes versus numerical ones.

A thing I learned about CBR that it is best utilized when data can be changed constantly. Because you can add new cases as experiences accumulates and use the most similar ones.